

Department of Computer Science and Engineering, IIT Palakkad

CS 2030 Data Structures and Algorithms

End-semester Exam (7 May 2024)

Time: 9:30 hrs to 12:30 hrs

Points: 50

Name: ... *Knithike*

Roll No:

Instructions

1. This quiz-booklet has 14 pages containing 10 questions. Please get a new booklet if any question/page is missing.
2. Write your **name** and **roll number** in the space above.
3. Answer each question in the corresponding space provided. **Additional sheets may be used only for rough work and will not be evaluated.**
4. Write your answers neatly in blue or black ink. Do not use pencil or red ink.
5. Do not keep any books, papers, mobile phones or other smart gadgets with you during the exam.
6. All answers require adequate justifications.

2

1. Let $f(n)$ and $g(n)$ be asymptotically positive functions. State true or false for each of the following statements and justify your answer.

(a) $f(n) \in O(g(n))$ implies $2^{f(n)} \in O(2^{g(n)})$.

1 mark

False. Counter ex: $2n \in O(n)$
 $2^{2n} \notin O(2^n)$

(b) $f(n) \in \Theta(f(n/2))$.

1 mark

False. Counter ex: $2^n \notin \Theta(2^{n/2})$

3

2. In each of the following parts, indicate whether $f \in \Theta(g)$ or not. If $f \notin \Theta(g)$, then specify if $f \in O(g)$ or $f \in \Omega(g)$. Assume 2 as the base of logarithmic functions.

no justification
 reqd

(a) $f(n) = 10 \log n$ and $g(n) = \log n^{100}$.

1

$f(n) \in \Theta(g(n))$

(b) $f(n) = n^{100}$ and $g(n) = 2^{\log^2 n}$. Recall that $\log^k n$ denotes $(\log n)^k$.

1

$f(n) \in O(g(n))$ $2^{\log^2 n} = (2^{\log n})^{\log n} = n^{\log n}$

(c) $f(n) = \sqrt{n}$ and $g(n) = 2^{\sqrt{\log n}}$.

1

$f(n) \in \Omega(g(n))$ $\sqrt{n} = 2^{\log n^{1/2}}$

5

3. Consider a doubly linked list with the following operations.

- **Insert(L.head, i, x)**: inserts x into L at index i and returns a pointer to the first element in L .
- **Delete(L.head, x)**: deletes the first occurrence of x in L (if it is present) and returns a pointer to the first element in L .
- **Find(L.head, x)**: returns a pointer to the first occurrence of x in L if it is present, otherwise returns NULL.

Give pseudo-codes for the following operations using only the above operations. Analyze the time complexity for each of the operations assuming that each of the above operations take $O(1)$ time. Arguments justifying the correctness are not required.

- **Find2(L.head, x)**: returns a pointer to the second occurrence of x in L if it exists, otherwise returns NULL.
- **RemoveDuplicate(L.head, x)**: deletes all duplicates of x from L and returns a pointer to the first element in L .

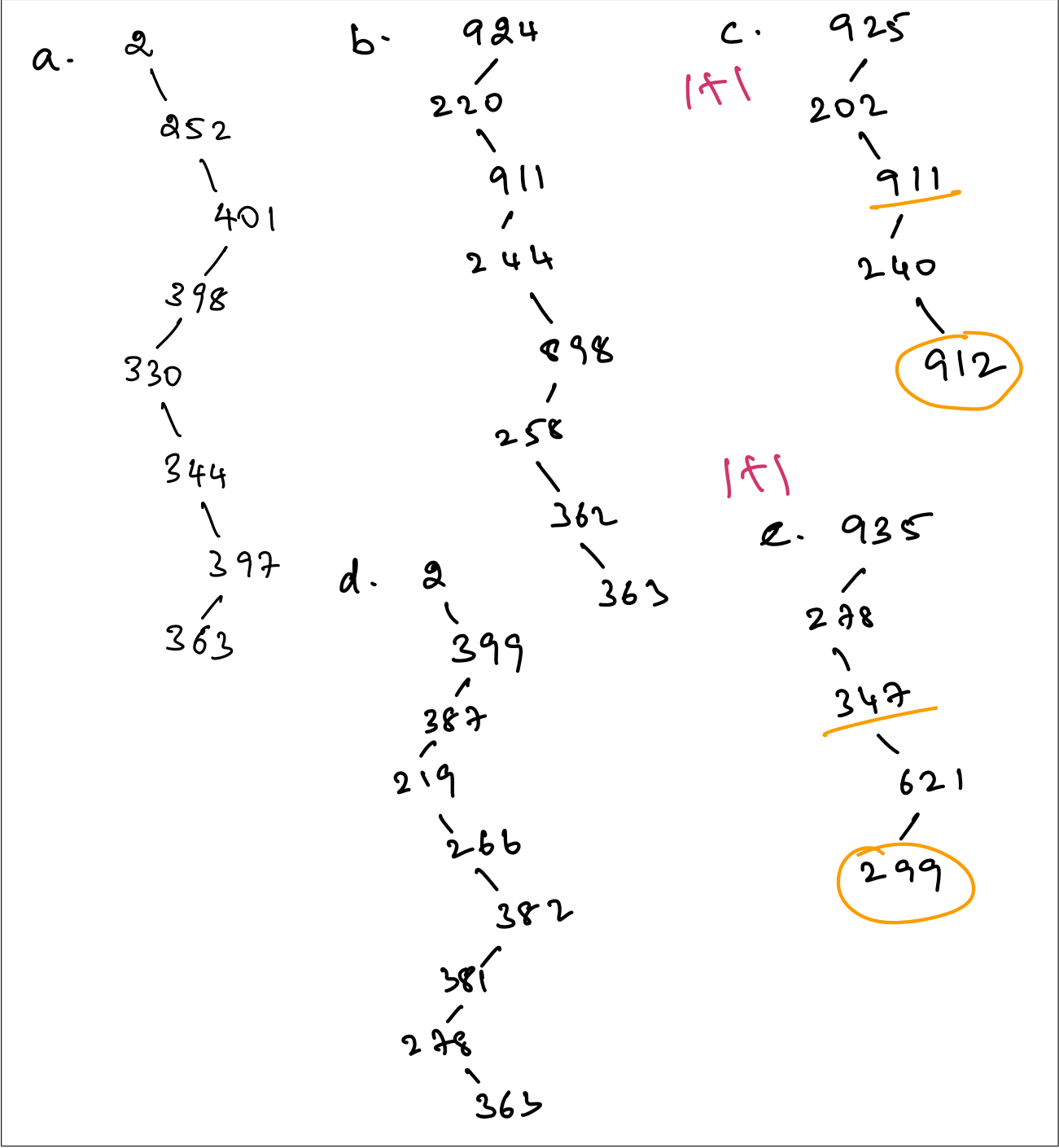
Find2(L, x) If del $\rightarrow$ 0 marks
 z = Find(L, x) If null not checked
 If z = null say x not found -1/2
 temp = Find(z.next, x) If first occ ret mark
 If temp = null say no sec occ of x
 else return temp

RemoveDuplicate(L, x) If 1st occ del
 or all occ del
 0 marks
 temp = Find2(L, x)
 while temp $\neq$ null
 delete(temp, x)
 temp = Find2(L, x)
 Return L

4. Suppose that we have numbers between 1 and 1000 in a binary search tree, and we want to search for the number 363 using the algorithm discussed in class. Which of the following sequences could not be the sequence of nodes examined? Justify your answer.

- a. 2, 252, 401, 398, 330, 344, 397, 363.
- b. 924, 220, 911, 244, 898, 258, 362, 363.
- c. 925, 202, 911, 240, 912, 245, 363.
- d. 2, 399, 387, 219, 266, 382, 381, 278, 363.
- e. 935, 278, 347, 621, 299, 392, 358, 363.

2 marks
2 marks



5. Where in a min-heap might the largest element reside, assuming that all elements are distinct? Justify your answer.

4

In a min heap with distinct keys,
a node has key val that is smaller
than any key val in its subtree.
∴ largest ele is a leaf.

6. Draw the expression tree corresponding to the expression $(a - (b/c)) * ((a/d) + e)$. Give the preorder and postorder traversal sequences of the resulting tree. 4

2 marks

Preorder: $* - a / b c + / a d e$ 1 mark

Postorder: $a b c / - a d / e + *$ 1 mark

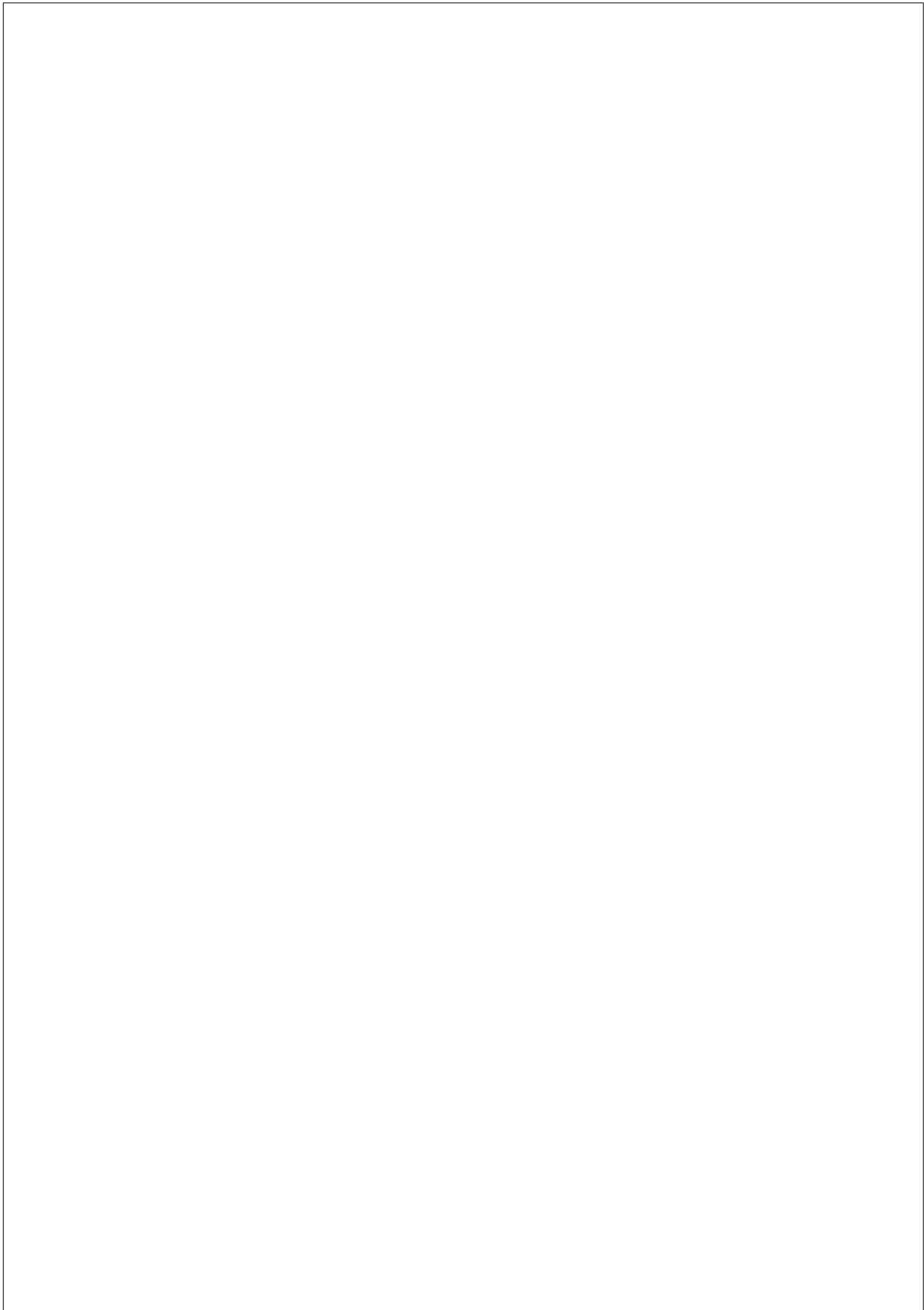
7. List the strongly connected components of the following graph G . Perform a depth-first search on G ; whenever there is a choice of vertices, pick the one that is alphabetically first. Give the start and finish times for each vertex. 5

3 marks

Start/finish

d h e f g

2 marks



8. When do you say that a family $\mathcal{H}$ of hash functions from $[N]$ to $[m]$ is a universal family? Suppose n keys are hashed into a table of size m using a random function from a universal family and collisions are resolved using the chaining method. How do you search for an element in the table? Calculate the expected time for an unsuccessful search.

A family $\mathcal{H}$ of hash functions is *universal* if $\forall x, y \in U$ s.t. $x \neq y$

2 marks $|\{h \in \mathcal{H} : h(x) = h(y)\}| \leq \frac{|\mathcal{H}|}{m}$

i.e. $\Pr_{h \in \mathcal{H}}(h(x) = h(y)) \leq \frac{1}{m} \quad \forall x, y \in U, x \neq y$

universal
choose $h \in_p \mathcal{H}$ and use h to hash S into T

1 mark Search in $T[h(x)]$
list corresp to $h(x)$

Expected Time to Search for $x \in U \setminus T$ 3 marks

Define r.v $C_{x,y} = 1$ if $h(x) = h(y)$
 $= 0$ o/w

Let $L_x = \sum_{y \in S} C_{x,y}$ = no. of ele of S hashed to same slot as x
 $\downarrow$ dataset
 T

$$E[C_{x,y}] = \Pr(h(x) = h(y)) = \frac{1}{m}$$

$$E[L_x] = \sum_{y \in S} E[C_{x,y}] \leq \frac{n}{m} = \alpha \text{ load factor}$$

$\therefore$ expected time for unsuccessful search is $O(1 + \alpha)$.

As long as α is a constant, the expected time for unsuccessful search is a constant.

Further, by Markov's inequ,

$$\Pr_{h \in \mathcal{H}}(L_x \geq 2E[L_x]) \leq \frac{1}{2}$$

9. Recall that an undirected graph is a tree if it is connected and acyclic.

(a) Show that any tree on at least 2 vertices has two vertices of degree 1.

4

(b) Show that there is a unique path between any two vertices of a tree.

5

4 marks

(a) Consider a maximal path

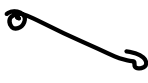
The end vertices of the path

(1) cannot be adj to other vertices of path (except its 1 nbr on path) o/w cycle

(2) cannot be adj to vertices outside the path o/w not maximal path

5 marks

(b) Induc on no. of vertices n

Base $n = 1$ $\bullet$ $n = 2$ 

Induc Hyp: Assume there is a unique path b/w any 2 vertices of a tree on k vertices, $k \geq 1$

Consider a tree T on $k+1$ vertices

Let v be a leaf in T + u be its nbr

$T-v$ is a tree on k vertices
 & by induc hyp any 2 vertices of
 $T-v$ has a uniq path b/w them

Suppose in $T \exists x, y$ s.t there
 are 2 paths b/w $x \neq y - P_1 \neq P_2$

Case 1: $x \neq v \neq y$
 v can't appear in $P_1 \neq P_2$ as v
 has deg 1 & internal vertices of
 $P_1 \neq P_2$ have deg ≥ 2

Then $P_1 \neq P_2$ are in $T-v$ leading
 to a contradiction

Case 2: $x = v$ or $y = v$
 without loss of generality let $x = v$
 Consider pred of v in $P_1 \neq P_2 \rightarrow u$
 Then we get 2 diff paths from y
 to u in $T-v$ leading to a contradiction

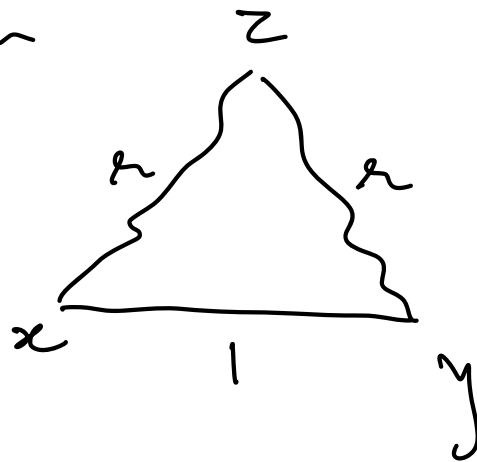
10. An undirected graph is called bipartite if it contains no odd length cycles.

- (a) Show that if G is bipartite then G has no edges between vertices in the same level of any BFS forest. 3
- (b) Given a connected bipartite graph G , modify BFS to find a partition of the vertex set into A and B such that each edge is between some vertex in A and some vertex in B . Argue correctness of the algorithm and analyze its running time. 5

Suppose $\exists x, y$ same level l
are adj

let z be common ancestor of x, y
with max depth

Then



odd cycle

BREADTH-FIRST SEARCH

$\text{BFS}(G, s)$ *arb vertex*

For each $v \in V(G)$ set $\text{pred}(v) = \text{null}$ and $d(v) = \infty$ // $d(v)$ is estimate of $\delta(s, v)$
 $d(s) = 0$

Initialize H as $V(H) = \{s\}, E(H) = \emptyset$ // BFS tree

Enqueue(Q, s) // To look at its neighbours

While Q is not empty

- $v = \text{Dequeue}(Q)$
- For each $(v, x) \in E(G)$
 - If $d(x) > d(v) + 1$ // when v was added to Q , $d(v)$ was updated
 - $d(x) = d(v) + 1$ and $\text{pred}(x) = v$
 - Add x to $V(H)$ and (v, x) to $E(H)$
 - Enqueue(Q, x)

Return d, H *(A, B)*

3 marks

*Initialize $A, B = \emptyset$
for each $v \in V(G)$*

*if $d(v) \% 2 = 0$
Add v to A*

else Add v to B

► Overall running time is $\mathcal{O}(n + m)$ and H is an out tree.

► Upon termination of BFS on (G, s) , $\forall v \in V(G)$, $d(v) = \delta(s, v)$. Also, H is a s -SPT.

Correctness : 1 mark

